Ders 3 Genişletilmiş Notu: Birliktelik Kuralları ve Apriori Algoritması

Ders 3 Genişletilmiş Notu: Birliktelik Kuralları ve Apriori Algoritması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Akışı
- 2. Birliktelik Kuralları Nerede Kullanılır?
- 3. Bir Kuralın Kalitesi: Destek ve Güvene İlk Bakış
- 4. Notasyon ve Merkez Ofis Senaryosu
- 5. Dokümanlarda Birliktelik: Bag of Words
- 6. Birliktelikten Kurala
- 7. Destek ve Güven: Tanımlar ve Formüller
- 8. Problem Tanımı: Minimum Destek ve Minimum Güven
- 9. Elle Çözülen İki Örnek
- 10. İşlem Verisinin Biçimi
- 11. Apriori'nin İki Aşaması ve Aşağı Kapalılık Özelliği
- 12. Apriori'nin Adım Adım Çalıştırılması
- 13. Algoritmanın Sözde Kodu
- 14. Aday Üretimi: Birleştirme ve Budama
- 15. Sık Kümelerden Kural Üretimi
- 16. Apriori'nin Maliyeti
- 17. Tablo Verisini İşlem Biçimine Çevirme
- 18. Sınıf Birliktelik Kuralları Madenciliği
- 19. Sıralı Örüntü Madenciliğine Giriş
- 20. Soru-Cevap: Sınıf Birliktelik Kuralları ve TF-IDF

1. Dersin Akışı

Bu dersin konusu birliktelik kuralı çıkarma algoritmalarıdır (association rule mining). Hoca, bunların web madenciliğinde nerede kullanıldığından ve endüstride karşılaşılan problemlerden de bahsedeceğini söyledi. Veriden birliktelik ve kural çıkarmak, endüstride çok ihtiyaç duyulan bir iştir.

Dersin planlanan akışı şöyledir:

- Birliktelik kuralları nedir?
- Bunun için kullanılan temel algoritma olan Apriori nasıl çalışır?
- Apriori'nin farklı işlem verileri için kullanılan bir sürümü olan sınıf birliktelik kuralları madenciliği (class association rule mining) nedir?
- Sıralı örüntü madenciliği (sequential pattern mining) nedir?

> **Hocanın vurgusu:** Ders, kitabı bölüm bölüm izliyor. Slaytlarda işlenen konuların kitapta karşılığı var. Her bölümden sonra ilgili kitap bölümünü, içindeki ayrıntılı örnekleri ve bölüm sonundaki soruları mutlaka okuyup çalışın.

2. Birliktelik Kuralları Nerede Kullanılır?

2.1 Market Sepeti ve İşlem Kavramı

Birliktelik kuralları özellikle müşteri işlemleri (transaction) üzerinde kullanılır. Hocanın örneği süpermarkettir. Müşteri Migros'ta ya da Carrefour'da alışverişini yapar, kasaya gider ve sepetindeki bütün ürünlerin parasını tek seferde öder. Fişin üzerindeki bütün ürünler aynı anda satın alınmış olur. İşte buna bir işlem diyoruz. Demek ki bir işlemin içinde eş zamanlı alınan ürünler vardır.

Birliktelik kuralı çıkarma algoritmaları çok sayıda insanın işlem verisine bakar. Bu işlemlerde birlikte tekrar etmiş popüler birliktelikleri bulur. Siz gidip süt ve ekmek aldınız diyelim. Bu ikiliyi milyonlarca kişi de işlemlerinde gerçekleştirmişse, en sık gerçekleşen satın almalar listesinde süt ve ekmek birlikte çıkar.

2.2 Çapraz Satış

Popüler birliktelikleri bulmamızın bir nedeni çapraz satıştır (cross-sell). Bu, pazarlamadan gelen bir kavramdır. Popüler bir ürünü satarken yanına daha az popüler bir ürünü de satmaktır. Müşterinin ikinci ürünü alma niyeti çok yüksek değildir, ama popüler ürün sayesinde onu da alabilir. Kısacası satılma potansiyeli düşük bir ürün, satılma potansiyeli yüksek bir ürünle birleştirilir ve böylece o da satılmış olur.

Bunu e-ticaret sitelerinde çok yoğun görüyoruz. Migros ve Carrefour gibi fiziksel mağazaları (retail store) olan yerlerde de görüyoruz.

2.3 Üst Satış

İkinci kullanım üst satıştır (up-sell). Bir ürünün farklı seviyeleri olabilir. Bunu bir hizmet ürünü gibi düşünün. Bir yerde orta seviye bir üyeliğiniz var. Firma önce bu üyeliği satmış. Sonra reklam yoluyla ya da kişinin hizmetten memnuniyeti sayesinde bu üyeliği bir üst seviyeye yükseltmiş. Bu durum işlemlerde görünür.

Bu yükseltme çok sayıda insanın işleminde popüler biçimde geçiyorsa, ürünü satarken yükseltilmiş sürümünü de pazarlayabilirsiniz. Hocanın örnek söylemi şudur: "Bu ürünü alan müşterilerimizin %90'ı sonradan yükseltmesini de almıştır, dolayısıyla size de bunu önerelim."

Çapraz satış ve üst satış yapılabilecek ürünleri tespit etmek için birliktelik kuralı çıkarma algoritmalarını kullanırız.

2.4 Problemin Tarihi ve Bugünkü Araçlar

Bu problem yeni değildir. Fiziksel mağazası olan perakendecilerde çoktandır vardır. Problemi çözmek için ilk algoritma 1993 yılında önerildi. Probleme de market sepeti analizi (market basket analysis) problemi dendi. Bunu popüler birliktelikleri ortaya çıkarma problemi diye tanımlayabiliriz.

İlk algoritmayı Agrawal önerdi. Bu çalışmada verilerin sayısal değil kategorik olduğu varsayıldı.

Konu çok popüler olduğu için Apriori'den sonra çok sayıda farklı algoritma önerildi. Bu algoritmalar bugün Weka'da, Python kütüphanelerinde, Spark ve Apache Flink gibi büyük veri işleme platformlarının makine öğrenmesi kütüphanelerinde bulunuyor.

> **Ders sonrası not:** Hoca, Apriori'yi Agrawal'ın 1993'te önerdiği algoritma olarak anlattı. Problemin ilk tanımı 1993 tarihlidir. Apriori adıyla bilinen algoritma ise büyük olasılıkla Agrawal ve Srikant'ın 1994 tarihli çalışmasıdır.

3. Bir Kuralın Kalitesi: Destek ve Güvene İlk Bakış

Slaytta ekmek (bread) ve sütün (milk) çok sayıda işlemde birlikte çıktığı bir örnek var. Bundan bir kural elde ediliyor. Böyle bir kuralın kaliteli olup olmadığını anlamak için iki ölçüt kullanılır: destek (support) ve güven (confidence).

Destek, kuralın popülerliğini ifade eder: İşlemlerin yüzde kaçında bu ikili birlikte geçiyor? Hocanın örneği:

- Verilen: Toplam 100 işlem var. Bunların 5'inde ekmek ve süt birlikte geçiyor.
- İşlem: $5/100$.
- Sonuç: Destek %5 (ara adım eklendi).

Güven, kurala güvenip güvenemeyeceğimizi söyler: İçinde ekmek olan işlemlerin yüzde kaçında süt de var? Hocanın örneği:

- Verilen: 100 işlemin 5'inde ekmek var. Bu 5 işlemin 5'inde de süt var.
- İşlem: $5/5$.
- Sonuç: Güven %100. Yani bu kurala güvenimiz %100'dür.

4. Notasyon ve Merkez Ofis Senaryosu

Algoritmayı kullanırken veriyi nasıl modelleyeceğimizi bilmemiz gerekir. Notasyon şöyledir:

- I: Üzerinde çalıştığımız bütün öğelerin (item) kümesi. Slaytta m tane öğe var.
- t: Tek bir işlem. Carrefour'a gidip alışveriş yaptınız ve kasada ödeme yaptınız. Bu sizin için bir işlemdir. Üç ürün aldıysanız işlemin içinde üç ürün vardır.
- T: Farklı kişilerin yaptığı bütün işlemleri içeren veritabanı. Slayttaki örnekte n tane işlem var ve her işlemde farklı sayıda ürün olabilir.

Her işlemin mutlaka bir işlem kimliği (transaction ID) vardır.

> **Ders sonrası not:** Hoca I'nın alt elemanlarından söz ederken "subset'leri item'ları gösteriyor" dedi. Doğrusu: Her öğe I'nın bir elemanıdır. Bir işlem ise I'nın bir alt kümesidir.

Hoca bu notasyonu perakende senaryosuyla somutlaştırdı. Her gün insanlar en yakın süpermarkete gidiyor ve işlemler oluşuyor. Günlük işlemler büyük işlem veritabanında toplanıyor. Carrefour'un merkez ofisi, bütün mağazalardan gelen işlemler üzerinde birliktelik kuralı çıkarma algoritmalarını çalıştırıyor. En yaygın olarak birlikte satılan ürünleri tespit ediyor.

Bundan sonra içlerinden birlikte satılabilecek olanları birleştiriyor ya da kasadaki görevlinin hemen yanına, görünür bir yere koyuyor. Alışverişinizi tamamladığınızda görevli size o ürünleri gösterip "bunlarda indirim var" diyerek öneride bulunuyor. Aslında o ürünler önceden üzerinde çalışılmış, çok sayıda insana satıldığı belli olmuş ürünlerdir.

5. Dokümanlarda Birliktelik: Bag of Words

İşlemlerden birliktelik çıkarma o kadar popüler oldu ki veri alanında çalışanlar bunu başka yerlerde de kullanıp kullanamayacaklarını sordular. Yanıtlardan biri, yine web madenciliği alanında, dokümanlar üzerinde bulundu.

Elinizde çok sayıda doküman olsun. Sizden bu dokümanlarda en çok tekrar eden kelime gruplarını bulmanız istensin. Her dokümanı bag of words (kelime torbası) biçiminde gösterebilirsiniz. Bunun için dokümandaki bağlaçları, şaşkınlık ve ünlem ifade eden kelimeleri çıkarırsınız. Kalan anlamlı kelimeleri bir araya getirerek dokümanı temsil eden bir vektör oluşturursunuz.

Doküman seti üzerinde birliktelik kuralı algoritmalarını çalıştırınca, dokümanların büyük bölümünde tekrar eden kelime gruplarını bulursunuz. Yüksek destek seviyesinde bir birliktelik yakaladıysanız şunu diyebilirsiniz: Bu doküman seti ağırlıklı olarak şu konular üzerine yoğunlaşmış. Basketbol ve spor, futbol ya da politika gibi çıkarımlar yapabilirsiniz.

6. Birliktelikten Kurala

Hoca buraya kadar hep "birliktelik" dedi, yani beraber satılan ürünleri tespit etmekten söz etti. Şimdi "kural" kavramına geçiyoruz.

Tespit ettiğimiz birliktelik süt ve ekmek olsun. Bu ikiliden iki kural çıkar:

- Ekmek $\rightarrow$ süt: İnsanlar ekmek alırsa süt de alır.
- Süt $\rightarrow$ ekmek: İnsanlar süt alırsa ekmek de alır.

Amaç önce işlemlerdeki popüler birliktelikleri yakalamak, sonra bunlardan yola çıkarak kurallar çıkarmaktır.

Bir kural $X \to Y$ biçiminde yazılır. İki şart vardır:

$$X \subset I, \qquad Y \subset I, \qquad X \cap Y = \emptyset$$

Yani kuralın iki tarafı da bütün öğeler kümesinin alt kümesidir ve kesişimleri boştur. Bir ürün hem solda hem sağda olamaz. İki tarafta da tek öğe olması gerekmez. Örneğin sol tarafta ekmek ve çikolata, sağ tarafta süt ve mısır gevreği (cereal) olabilir.

Slaytta bir de öğe kümesi (itemset) örneği var: $X = \{\text{milk}, \text{bread}, \text{cereal}\}$. Üç elemandan oluştuğu için buna 3 öğeli küme (3-itemset) denir. Bu üç ürün bir kuralın sol tarafında, başka bir ürün de sağında olabilir. Bu durumda kural, üç ürünü alan bir kişinin sağ taraftaki ürünü de alacağını söyler.

> **Ders sonrası not:** Hoca bu kümede "3 tane kural olacak" dedi. Doğrusu 3 öğe. Altyazıdaki "Mint" kelimesi büyük olasılıkla "milk"tir.

Çıkarılan kurallar sıralı kullanılır. Diyelim 10 kural çıkardınız. Bunları belli bir sıraya göre yukarıdan aşağıya dizer, sonra çapraz satış ve üst satış politikalarınızı bu kurallara göre geliştirirsiniz. Peki kuralların önem değerini nasıl belirleyeceğiz? Destek ve güven ölçütleriyle.

7. Destek ve Güven: Tanımlar ve Formüller

7.1 Destek

Destek, toplam işlemlerin yüzde kaçında hem X'teki hem Y'deki öğelerin birlikte bulunduğunu söyler. Hoca aklımızda kalması için şöyle dedi: Bu veri seti bu kuralı ne kadar destekliyor?

$$\text{support}(X \to Y) = \frac{\text{count}(X \cup Y)}{n}$$

Burada n toplam işlem sayısıdır.

7.2 Güven

Güven, kurala ne kadar güveneceğimizi söyler. Bu kez kuralın sol tarafına bakarız. X'teki öğeleri içeren işlemlerin yüzde kaçı Y'deki öğeleri de içeriyor? Ekmek-süt örneğinde, ekmek içeren işlemlerin yüzde kaçında süt de var diye sorarız. Çünkü bazı insanlar ekmek alıp süt almamış olabilir.

$$\text{confidence}(X \to Y) = \frac{\text{count}(X \cup Y)}{\text{count}(X)}$$

Önce X'i içeren işlemlerin sayısı bulunur. Bu, paydadır. Pay ise bu işlemlerden hem X'i hem Y'yi içerenlerin sayısıdır. X'i içeren işlemlerin tamamında Y de varsa güven %100 olur.

Her kural için hem bir destek hem bir güven değeri bulmamız gerekir.

Soru: Bir önceki slaytta, güven hesabında yalnız Y'yi içerip X'i içermeyenler de var diye anlamamız gerekmiyor mu?

Cevap: Güven formülü kurala göre kurgulanır. $X \to Y$ kuralı için önce X'in bulunduğu işlemlere bakılır. Diyelim 100 işlemin 5'inde X var. Payda bu 5'tir. Şimdi bu 5 işlemin kaçında Y var diye sorulur. Belki 3'ünde, belki 5'inde, belki 1'inde. Kural $Y \to X$ olsaydı paydada Y'nin sayısı olurdu. Pay değişmezdi.

Soru: Ekmek ve süt örneğinde ekmek almayıp süt alan insanlar da olacak. Onlar hesaba giriyor mu?

Cevap: Hayır, güven hesabına katılmazlar. Kuralın sol tarafında ekmek var. Bu kural için bakış açımız önce ekmek almış olmaktır, yani bir ekmek alma şartı vardır. Yalnız süt alanları bu kural için saymayız. Formüldeki birleşim işaretinin kastı, hem ekmek hem süt almış olanları bulmaktır. Sonra bunların, yalnız sol taraftaki ürünü almış olanların (ekmek alanların) içindeki oranına bakılır.

8. Problem Tanımı: Minimum Destek ve Minimum Güven

Birliktelik kuralı çıkarma çalışmasında bize iki değer verilir: minimum destek (minimum support) ve minimum güven (minimum confidence).

Hocanın senaryosunda mağaza sahibi şöyle der: "Müşterilerimin satın alımlarında %50'nin üzerinde popülerlikle alınan ürünler varsa onlarla çapraz satışa çıkmak istiyorum. Son 6 aylık verilerde birlikte satılan ürünleri bulabilir misin?" Bunu söylerken aslında destek değerini söylemektedir: %50.

Devamında şunu da ister: "Bu birlikteliklerden kural da çıkarabilir misin?" Üç ürünün birlikte satıldığını bulduk. Peki bunların birlikte satılmasını hangi ürün tetikliyor? Mağaza sahibi "A ürünü satılırsa yüksek güvenle B ve C ürünleri de satılır" gibi kurallar ister ve bu kuralların güveninin de en az %50 olmasını şart koşar. Böylece destek de güven de %50 olarak belirlenmiş olur.

> **Ek bilgi (derste anlatılmadı):** Birliktelik kuralı, ürünlerin işlemlerde birlikte görülme sıklığını ölçer. Yüksek güven tek başına sol taraftaki ürünün sağ taraftakinin satılmasına neden olduğunu kanıtlamaz. "Tetikleme" yorumu bu yüzden bir iş varsayımı olarak düşünülmeli, nedensellik için ayrı bir sınama gerekir.

Bu iki değer verildikten sonra iş, mevcut işlem verisi üzerinde bu değerlere dayalı kuralları çıkarmaktır. Burada iki noktaya dikkat etmeliyiz:

- Algoritma, verideki olabilecek kuralların tamamını bulmalıdır. Hiçbir kural atlanmamalıdır.
- Kuralın hem sol hem sağ tarafında ürün olmalıdır.

9. Elle Çözülen İki Örnek

9.1 Beş İşlemlik Örnek

Slaytta 5 işlemlik bir tablo var. İstenen, önce birlikte satılan ürünleri, yani sık öğe kümelerini (frequent itemsets) bulmak, sonra bunlardan kural çıkarmaktır. Sık öğe kümesi, çok sayıda farklı işlemde beraber geçen popüler ürün grubudur. Bu örnek bir önceki bölümdeki %50 destek ve %50 güven değerleriyle gözle çözülüyor.

Verilen: $n = 5$, minimum destek %50, minimum güven %50. Yani bir kümenin sık olması için $0{,}5 \times 5 = 2{,}5$ olduğundan en az 3 işlemde geçmesi gerekir (ara adım eklendi).

Adım 1, tekli sık öğeler: Hocanın saydığı değerler:

Öğe	Geçtiği işlem sayısı
A	3
B	3
D	4
E	3

Adım 2, ikili sık küme: $\{A, D\}$, 5 işlemin 3'ünde geçiyor. Hocanın burada adını verdiği ikili budur.

Adım 3, kural üretimi: Tek elemanlı birlikteliklerden kural çıkmaz. Kural için en az iki elemanlı birliktelik gerekir. $\{A, D\}$'den iki kural çıkar: $A \to D$ ve $D \to A$.

Adım 4, destek: Destek, kuralın iki tarafındaki öğelerin toplam işlemlerde birlikte kaç kez geçtiğine bakar. Bu yüzden iki kuralın desteği aynıdır:

$$\text{support}(A \to D) = \text{support}(D \to A) = \frac{3}{5} = \%60$$

Adım 5, güven:

- $A \to D$: A içeren işlem sayısı 3. Bunların 3'ünde D var. Güven $3/3 = \%100$.
- $D \to A$: D içeren işlem sayısı 4. Bunların 3'ünde A var. Güven $3/4 = \%75$.

Sonuç: İki kuralın güveni de en az %50 şartını sağlıyor. Dolayısıyla iki kural da çıkarılır.

Hoca bu çözümdeki yaklaşıma dikkat çekti: Önce tekliler, sonra ikililer, sonra üçlüler bulunur. Sonra ikililerden başlanarak kurallar çıkarılır. Gözle yapılan bu işi Apriori bir algoritma içinde yapar.

> **Ders sonrası not:** Hoca $A \to D$ için "desteği %100, güvenilirliği %100" dedi. Doğrusu: destek %60, güven %100. Destek değerini kendisi de az önce %60 olarak hesaplamıştı.

9.2 Yedi İşlemlik Örnek

İkinci örnekte 7 işlem var. İstenen, desteği %30 ve güveni %80 olan kurallardır. Hoca tekli ve ikili aşamaları geçip doğrudan sonuca gitti: En fazla elemanı olan popüler öğe kümesi 3 elemanlıdır.

Verilen: $n = 7$, minimum destek %30, minimum güven %80.

Sık küme: $\{\text{chicken}, \text{clothes}, \text{milk}\}$. 7 işlemin 3'ünde geçiyor. Destek:

$$\frac{3}{7} \approx \%42{,}9$$

Bu değer %30 eşiğinin üstündedir.

> **Ders sonrası not:** Hoca "3 bölü 7 %30 support değerime karşılık geliyor, ondan yukarıda" dedi. Doğrusu: $3/7 \approx \%42{,}9$ ve bu değer %30 eşiğinin üstündedir.

Kural yazımı: Önce sol tarafa bir öğe konur, kalan iki öğe sağa yazılır. Bu her öğe için yapılır. Sonra ikililer sol tarafa konur ve kalan öğe sağa yazılır. Sonra hepsinin güveni hesaplanır.

Bir güven hesabı: $\text{clothes} \to \{\text{milk}, \text{chicken}\}$ kuralına bakalım. Clothes içeren 3 işlem var. Bunların 3'ü milk ve chicken'ı da içeriyor. Güven $3/3 = \%100$.

Gözlem: Bu kuralların hepsi aynı üç öğeyi içerdiği için destekleri kümenin desteğiyle aynıdır, yani değişmez. Yalnız güven değerleri her kural için ayrı hesaplanır.

10. İşlem Verisinin Biçimi

Apriori'nin ayrıntısına geçmeden önce hoca işlem verisinin biçimini anlattı. En basit işlem veritabanında yalnız işlem kimliği ve ürün kimlikleri vardır. Slaytta ürün adları yazılı, ama normalde ürünlerin kimlikleri olur.

Gerçek işlem verilerinde ürün kimliğinin yanında, parantez içinde ya da bir ayraçla, ürünün fiyatı ve adedi de bulunabilir. Algoritmayı çalıştırırken bunu basitleştiriyoruz. Ürünü fiyatından ve adedinden arındırıp yalnız kimliğini kullanıyor ve popüler birliktelikleri bu veritabanına bakarak buluyoruz.

Bir işlem veritabanında çok sayıda işlem, işlemlerde de çok sayıda ürün olabilir. Biz önce birliktelikleri, sonra bu birlikteliklerden birliktelik kurallarını çıkaracağız. Bunun için kullanacağımız algoritma Apriori'dir.

11. Apriori'nin İki Aşaması ve Aşağı Kapalılık Özelliği

11.1 İki Aşama

Apriori, işlem verisi üzerinde popüler birliktelikleri iki aşamada bulur:

1. Sık öğe kümelerini bulur. Bir elemanlı, iki elemanlı, üç elemanlı kümeler olabilir.
2. Bulduğu sık kümeleri kullanarak kurallar üretir.

Algoritmanın girdisi veri setinin kendisi, minimum destek ve minimum güven değerleridir. Hocanın slayttaki örneğinde birinci aşama bir üçlü bulmuş. Bu üçlünün desteği problemin başındaki destek değerini sağlıyor. İkinci aşama da bundan bir kural çıkarıp güven değerini bulmuş.

> **Ders sonrası not:** Hoca hem "olabilecek bütün frequent item setlerini buluyor" dedi hem de algoritmanın "en fazla elemanlı" kümeyi bulup kuralları ondan çıkardığını söyledi. Standart anlatımda çıktı büyük olasılıkla bütün sık kümelerdir ve kurallar her birinden üretilir. Kurallar yalnız en büyük kümeden yazılırsa daha küçük bir sık kümenin kuralları kaçar. Örneğin $X = \{2, 3, 4\}$'ten yazılan her kuralın iki tarafının birleşimi X'tir, bu yüzden $\{2\} \rightarrow \{3\}$ bu listede yoktur. Bu da "hiçbir kuralı atlamayalım" şartıyla çelişir.

11.2 Bir Kümenin Sık Olması

Apriori bir kümenin sık olup olmadığına karar verirken iki şeye bakar:

- **Minimum destek:** Kümenin veride geçme sıklığı problemde verilen minimum destek değerini sağlamalıdır.
- **Aşağı kapalılık özelliği (downward closure property):** Buna Apriori özelliği (Apriori property) de denir. İkisi aynı şeydir. Bir kümeye sık diyebilmek için bütün alt kümelerinin de sık olması gerekir.

Hoca bunu slayttaki bir örnekle gösterdi. Üst seviyede ABC, ABD, ACD ve BCD yazıyor. Bunlar alt seviyedeki ikililerin bir araya gelmesiyle oluşabilecek bütün seçeneklerdir. ABD'nin alt kümeleri olarak AB, AD ve BD'yi saydı. ABD'ye sık diyebilmek için hem ABD'nin kendi geçme sıklığı minimum desteği sağlamalı, hem de bu alt kümeleri popüler olmalıdır.

> **Hocanın vurgusu:** Algoritma iki şeye bakıyor: minimum destek değerinin sağlanması ve aşağı kapalılık özelliğinin sağlanması.

> **Ders sonrası not:** Hoca bunları iki ayrı şart olarak anlattı. Oysa destek şartı tek başına yeterlidir. X'i içeren her işlem X'in bütün alt kümelerini de içerir. Bu yüzden her alt kümenin desteği en az X'in desteği kadardır. Alt küme kontrolü ikinci bir tanım değil, sayım yapmadan aday elemeye yarayan bir kestirmedir.

12. Apriori'nin Adım Adım Çalıştırılması

Hoca algoritmayı önce bir örnek üzerinde çalıştırdı, sözde koda sonra geçti. Elimizde 4 işlem ve 5 farklı öğe var. İstenen, minimum destek %50 ile sık öğe kümelerini bulmaktır.

Önce çalışma biçimi: Algoritma yinelemeli (iterative) çalışır, yani bir döngü içindedir. Her turda veritabanını bir kez tarar. Veritabanını işlemlerden oluşan büyük bir tablo gibi düşünün. Algoritma bu tabloyu birinci satırdan son satıra kadar okur ve okurken elindeki öğelerin sayısını (count) artırır. Ayrıca aşağıdan yukarıya (bottom-up) çalışır, yani bir elemanlı kümelerden başlar.

Eşik: $0{,}5 \times 4 = 2$. Bir kümenin sık olması için en az 2 işlemde geçmesi gerekir.

12.1 Birinci Tarama: C_1 ve F_1

Algoritma tabloyu baştan sona tarar. Her satırda 5 öğenin sayaçlarını günceller. Tarama bitince bütün öğelerin sayıları elde edilmiş olur. Bu ilk listeye aday (candidate) listesi denir. Sayısı 2 ve üzeri olanlar tutulur. Bu da sık tekli kümeleri, yani F_1'i verir. Dört eleman kalır:

$$F_1 = \{1\},\ \{2\},\ \{3\},\ \{5\}$$

12.2 İkinci Tarama: C_2 ve F_2

Algoritma F_1'in elemanlarını birleştirerek (join) oluşabilecek bütün ikilileri listeler. Bu, sonraki taramanın aday listesidir:

$$C_2 = \{1,2\},\ \{1,3\},\ \{1,5\},\ \{2,3\},\ \{2,5\},\ \{3,5\}$$

Sonra veritabanının başına döner. Her satırda bütün adaylara bakar: "1 ve 2 var mı? 1 ve 3 var mı? 1 ve 5 var mı?" Varsa sayıyı artırır, sonra sonraki satıra iner. Yani yalnız bir ikilinin sayısını izlemez, her satırda bütün adayları günceller.

Tarama bitince sayısı 2 ve üzeri olanlar tutulur, diğerleri atılır. Hocanın anlatımında sık olarak geçen ikililer şunlardır:

$$F_2 = \{1,3\},\ \{2,3\},\ \{2,5\},\ \{3,5\}$$

12.3 Üçüncü Tarama: C_3 ve F_3

Veritabanını yeniden taramadan önce sayılacak aday listesi gerekir. Algoritma ikilileri birleştirip üçlüler oluşturur. Hocanın saydığı üçlüler arasında $\{1,2,3\}$, $\{1,3,5\}$ ve $\{2,3,5\}$ var.

Ama slaytta yalnız $\{2,3,5\}$ görünüyor. Bunun nedeni aşağı kapalılık özelliğidir. Sayıma geçmeden önce her adayın bütün alt kümelerinin bir önceki seviyede sık bulunup bulunmadığına bakılır:

Aday	İkili alt kümeleri	F_2'de olmayan	Karar
$\{2,3,5\}$	$\{2,3\},\{2,5\},\{3,5\}$	yok	Tutulur
$\{1,2,3\}$	$\{1,2\},\{1,3\},\{2,3\}$	$\{1,2\}$	Elenir
$\{1,3,5\}$	$\{1,3\},\{1,5\},\{3,5\}$	$\{1,5\}$	Elenir

Hoca $\{2,3,5\}$ için üç alt kümeyi tek tek kontrol etti ve diğer üçlülerin alt kümelerinin tamamının listede olmadığını söyledi. Tablodaki $\{1,2,3\}$ ve $\{1,3,5\}$ satırları bu söylemden kuruldu (ara adım eklendi).

Son olarak veritabanının başına dönülür ve $\{2,3,5\}$ sayılır:

İşlem satırı	$\{2,3,5\}$ var mı?	Sayaç
1	Yok	0
2	Var	1
3	Var	2
4	Yok	2

Sayı 2'dir. Eşik de 2'dir ($2/4 = \%50$). Algoritma $\{2,3,5\}$'i sık küme olarak bulur.

> **Ders sonrası not:** Hoca "1-3 ile 2-5'i merge ediyor, işte 1, 3, 5" dedi. Bu iki kümenin birleşimi $\{1,2,3,5\}$'tir ve 4 elemanlıdır, üçlü aday vermez. Standart Apriori'de birleştirme büyük olasılıkla yalnız ilk $k-2$ elemanı aynı olan kümeler arasında yapılır. O zaman F_2'den yalnız $\{2,3\}$ ve $\{2,5\}$ birleşir ve tek aday $\{2,3,5\}$ çıkar. Sonuç değişmez, çünkü hocanın saydığı diğer üçlüler budamada elenmektedir.

> **Ders sonrası not:** Hoca sık küme kontrolünü "minimum support'un üzerinde" diye de anlattı. Doğrusu "eşit veya üzerinde"dir. Kendisi de bu örnekte "2 ve 2'den fazla olanlar" diyor, kural üretimi örneğinde de "eşit ve üzerinde" diye düzeltiyor.

13. Algoritmanın Sözde Kodu

Hoca aynı süreci önce kısa, sonra daha ayrıntılı bir sözde kod (pseudocode) slaytı üzerinden, biraz önceki örnekle eşleştirerek anlattı. Kodun satırları altyazıda okunmuyor, ama akış şöyledir:

1. Girdi olarak veri seti ve minimum destek alınır.
2. Veritabanı bir kez okunur, bütün öğelerin sayıları hesaplanır. Bu, C_1 adımıdır.
3. Sayısının toplam işlem sayısına oranı minimum desteği sağlayan öğeler ayrılır. Bu, F_1'dir.
4. $k = 2$'den başlayan döngüye girilir:
    1. Önceki seviyenin sık kümeleri F_{k-1} ile aday üretme fonksiyonu çağrılır ve C_k bulunur.
    2. Veritabanı baştan sona okunur. Her satırda her adayın o satırda bulunup bulunmadığına bakılır, varsa sayısı artırılır.
    3. Sayısının n'ye oranı minimum desteği sağlayan adaylar seçilir. Bu, F_k'dir.
    4. Döngünün başına dönülür ve k bir artırılır.

Örnekle eşleşmesi açıktır. Döngüye girmeden yapılan iş C_1 ve F_1'dir. $k = 2$ turu C_2 ve F_2'yi, $k = 3$ turu C_3 ve F_3'ü verir.

> **Ek bilgi (derste anlatılmadı):** Döngü, önceki seviyede sık küme kalmayınca ya da aday üretilemeyince biter. Örnekte tek sık üçlü $\{2, 3, 5\}$ olduğundan birleştirecek ikinci bir üçlü yoktur ve dörtlü aday çıkmaz. Bu yüzden dördüncü bir tarama gerekmez.

14. Aday Üretimi: Birleştirme ve Budama

Sözde koddaki aday üretme fonksiyonu (candidate-gen) iki adımdan oluşur:

- **Birleştirme adımı (join step):** Teklileri ikiliye, ikilileri üçlüye çevirir.
- **Budama adımı (prune step):** Aşağı kapalılık özelliğini sağlamayan adayları atar.

Hoca fonksiyonu sık üçlülerden oluşan bir liste üzerinde gösterdi. Anlatımda adı geçen üçlüler şunlardır:

$$F_3 = \{1,2,3\},\ \{1,2,4\},\ \{1,3,4\},\ \{1,3,5\},\ \{2,3,4\}$$

Birleştirme: Üçlüler birbiriyle birleştirilip olası dörtlüler bulunur.

Birleştirilen üçlüler	Oluşan dörtlü
$\{1,2,3\}$ ve $\{1,2,4\}$	$\{1,2,3,4\}$
$\{1,2,4\}$ ve $\{2,3,4\}$	$\{1,2,3,4\}$
$\{1,2,3\}$ ve $\{1,3,5\}$	$\{1,2,3,5\}$
$\{1,3,4\}$ ve $\{1,3,5\}$	$\{1,3,4,5\}$

Hoca derste slaytta $\{1,2,3,5\}$'i göremedi ve kendisi ekledi. Böylece üç aday oluştu: $\{1,2,3,4\}$, $\{1,2,3,5\}$, $\{1,3,4,5\}$.

Budama: Fonksiyonun işi burada bitmez. Adayları döndürmeden önce her birinin üçlü alt kümelerinin hepsinin önceki seviyede sık olup olmadığına bakar.

- $\{1,2,3,4\}$: $\{1,2,3\}$ var, $\{2,3,4\}$ var, $\{1,3,4\}$ var. Hepsi sık, aday kalır.
- $\{1,3,4,5\}$: $\{1,3,4\}$ var, ama $\{3,4,5\}$ yok. Aşağı kapalılığı sağlamaz, listeden çıkarılır.
- $\{1,2,3,5\}$: $\{1,2,3\}$ var, ama $\{2,3,5\}$ yok. Listeden çıkarılır.

Üç adaydan yalnız biri kalır: $\{1,2,3,4\}$. Ama bu henüz sık küme değildir. Sonraki adımda veritabanının başına dönülür ve $\{1,2,3,4\}$'ün kaç işlemde geçtiği sayılır. Bir adayın iki özelliği sağlaması gerekir: aşağı kapalılık ve minimum destek. Budama birincisini, sayım ikincisini kontrol eder.

> **Ders sonrası not:** Hoca $\{1,2,3,4\}$ için üç alt küme saydı. Bu kümenin üçlü alt kümesi dört tanedir. Eksik olan $\{1,2,4\}$ de listede olduğu için sonuç değişmez.

Ders sonrası not: Standart birleştirme kuralında (ilk $k-2$ eleman aynı olmalı) büyük olasılıkla yalnız $\{1,2,3\}$-$\{1,2,4\}$ ve $\{1,3,4\}$-$\{1,3,5\}$ çiftleri birleşir. $\{1,2,3,5\}$ adayı o kuralla hiç oluşmaz. Hoca bunu ekledi, ama aday budamada zaten elendiği için sonuç yine $\{1,2,3,4\}$'tür.

Fonksiyonun genel davranışı şudur: Girdi sık tekliler, sık ikililer ya da sık üçlüler olabilir. Teklileri ikili, ikilileri üçlü listesine çevirir. Sonra alt kümeleri girdi listesinde olmayan adayları siler. Yani Apriori'nin döngüsündeki çağrı, her aday için aşağı kapalılığı kontrol ettikten sonra listeyi döndürür.

15. Sık Kümelerden Kural Üretimi

15.1 Yöntem

Apriori'nin sonunda elde edilen sık kümeler henüz birliktelik kuralı değildir. Bir aşama daha gerekir.

Bir sık küme X olsun. Hoca örnek olarak $X = \{1,3,4,5\}$'i verdi. Şu tanımları yapıyoruz:

- A: X'in boş olmayan bir alt kümesi. Bir ya da daha fazla eleman içerebilir.
- $B = X - A$: X'in A dışında kalan elemanları.

Artık $A \to B$ biçimindeki bütün kuralları yazabiliriz. X sık olduğu için bu kuralların hepsi minimum desteği sağlar, çünkü A ile B'nin birleşimi X'tir ve Apriori bu kümenin desteğinin istenen değerin üzerinde olduğunu zaten söylemiştir. Bu yüzden destek yeniden hesaplanmaz. Hesaplanması gereken yalnız güvendir:

$$\text{confidence}(A \to B) = \frac{\text{support}(A \cup B)}{\text{support}(A)} = \frac{\text{support}(X)}{\text{support}(A)}$$

Ders sonrası not: Hoca $\{1,3,4,5\}$'i "bir önceki örnekten" diye verdi. Bir önceki bölümde bu küme budamada elenmişti. Burada yalnız gösterim amaçlı bir örnek olarak okunmalıdır.

15.2 Örnek

Verilen: Apriori'nin bulduğu sık küme $X = \{2,3,4\}$, desteği %50. Yani işlemlerin %50'sinde 2, 3 ve 4 birlikte bulunuyor. Alt kümelerin destekleri de verilmiş. Hepsi minimum desteğin üzerinde ya da ona eşit.

Adım 1, kuralları yazma: A sırayla seçilir, kalanlar B'ye yazılır. Toplam 6 kural çıkar:

- $\{2,3\} \to \{4\}$
- $\{2,4\} \to \{3\}$
- $\{3,4\} \to \{2\}$
- $\{2\} \to \{3,4\}$
- $\{3\} \to \{2,4\}$
- $\{4\} \to \{2,3\}$

Adım 2, güven hesabı: Bütün kurallarda $A \cup B = \{2,3,4\}$ olduğu için pay her zaman %50'dir. Payda, kuralın sol tarafının desteğidir.

Kural	Pay	Payda: support(A)	Güven
$\{2,3\} \to \{4\}$	%50	%50	%100
$\{2,4\} \to \{3\}$	%50	%50	%100
$\{3,4\} \to \{2\}$	%50	%75	yaklaşık %67

Hoca, aşağıya doğru inerek bütün kuralların güveninin aynı şekilde bulunacağını söyledi.

Adım 3, eşik: Soruda "güveni %70'in üzerinde olanları yaz" denseydi yalnız ilk iki kural yazılırdı.

> **Ders sonrası not:** Tekli destekler altyazıda yok, ama kısmen çıkarılabilir. $\{3, 4\}$'ün desteği %75 olduğundan $\{3\}$ ve $\{4\}$'ün desteği en az %75'tir. Bu yüzden bu iki öğeyle başlayan kuralların güveni en çok $50/75 \approx \%67$ olur ve %70'in altında kalır. $\{2\}$ ile başlayan kuralın durumu ise ancak slayttaki değerle doğrulanabilir.

16. Apriori'nin Maliyeti

Apriori düzey düzey arama (level-wise search) yapar, yani aşağıdan yukarıya gider. Bir elemanlı kümelerle başlar, iki elemanlılara geçer. Önce adaylar bulunur, aşağı kapalılığı sağlamayanlar elenir, sayım yapılır ve gerçekten sık olanlar bulunur. Aynı iş üçlüler için tekrarlanır ve böyle yukarı çıkılır.

Her adımda veritabanı sıfırdan okunur. Dolayısıyla k elemanlı en büyük kümeyi bulmak istiyorsanız veritabanını k kez birinci satırdan son satıra kadar okumanız gerekir. Veritabanında milyonlar mertebesinde satır varsa bu maliyeti büyük bir algoritmadır.

Neden büyük? Çünkü diske her gidişiniz bir giriş-çıkış (I/O) işlemidir. Diskten okumanın maliyeti bellekten okumaya göre çok fazladır. Algoritma çok fazla diske giderse, yani k çok büyürse, oldukça fazla zaman harcar. Bu yüzden pratikte k için genelde bir üst sınır konur. Örneğin en fazla 10 elemanlı kümeler için çalıştırılır.

> **Hocanın vurgusu:** Algoritmanın kendisi hızlıdır. Pahalı olan, her seferinde veriye gidip onu okumaktır. Çünkü işlem veritabanları genelde satır sayısı çok fazla olan tablolardır.

17. Tablo Verisini İşlem Biçimine Çevirme

Veri farklı biçimlerde karşımıza çıkabilir. Apriori'nin görmek istediği biçim işlem biçimidir: Birinci satır birinci işlemi, ikinci satır ikinci işlemi gösterir.

Veri bazen tablo biçimindedir. O zaman Apriori'yi kullanabilmek için işlem biçimine çevirmek gerekir. Yöntem şudur: Her satır bir işlem kabul edilir. Sütun adı ile hücredeki değer ikili hale getirilir. Hocanın örneğindeki satır şöyle olur:

$$(\text{Attribute1}, a),\ (\text{Attribute2}, b),\ (\text{Attribute3}, d)$$

Böylece Apriori bu veriyi işleyebilir.

> **Ek bilgi (derste anlatılmadı):** Değerin sütun adıyla birlikte yazılmasının bir yararı da şudur: Aynı değer iki farklı sütunda geçse bile iki ayrı öğe sayılır. Böylece farklı özniteliklere ait değerler birbirine karışmaz.

18. Sınıf Birliktelik Kuralları Madenciliği

18.1 Fark Nedir?

Apriori'nin bir sürümü sınıf birliktelik kurallarını çıkarmak için kullanılır. Algoritma aynıdır. Tek fark üzerinde çalıştığı veridir: Normal Apriori'de sağ taraftaki etiketler yoktu ve yalnız elemanlar arasındaki sık birliktelikleri arıyorduk. Burada her işlemin bir etiketi vardır ve birliktelikleri sınıfıyla beraber buluruz. Kuralın sağ tarafına ürün yerine etiket konur.

> **Ders sonrası not:** Hoca başta "kuralın sol tarafında bir etiket var" dedi. Doğrusu sağ tarafıdır. Kendisi de devamında etiketi kuralın sağ tarafına koyuyor.

18.2 Örnek

Veri 7 dokümandan oluşuyor. Her satırda dokümanın kelimeleri ve etiketi (education ya da sport) var. Hoca etiketi önce sanki bir önceki algoritmadaki bir elemanmış gibi düşünmemizi söyledi.

Kural 1: $\{\text{student}, \text{school}\} \rightarrow \text{education}$

- Toplam işlem: 7.

- Student ve school içeren işlem: 2. Bunların 2'sinin etiketi education.
- Destek: $2/7 \approx \%28{,}6$ (yüzde karşılığı eklendi).
- Güven: $2/2 = \%100$.

Kural 2: $\{\text{game}\} \rightarrow$ sport

- Toplam işlem: 7.
- Game içeren işlem: 3. Bunların 2'sinin etiketi sport.
- Güven: $2/3 \approx \%66{,}7$ (yüzde karşılığı eklendi).
- Destek: Hoca bu değer için önce $2/7$ dedi, sonra kendini düzelterek $3/7$ dedi. Aşağıdaki nota bakın.

Hoca önce hem game hem sport içeren işlemleri saydı ve desteği $2/7$ buldu. Sonra bunu yanlış söylediğini düşünerek geri döndü. Sınıf birliktelik kurallarında desteğin yalnız sol taraftaki elemanların geçtiği işlem sayısının toplama oranı olduğunu söyledi. Buna göre ilk kural için $2/7$, ikinci kural için $3/7$ dedi.

> **Ders sonrası not:** Hocanın son söylediği tanım büyük olasılıkla yanlıştır, ilk söylediği doğrudur. Sınıf kuralının desteği, normal birliktelik kuralındaki gibi, hem X'i içeren hem etiketi y olan işlemlerin toplam işlemlere oranıdır: $\text{support}(X \rightarrow y) = \text{count}(X \cup \{y\})/n$. Buna göre $\{\text{game}\} \rightarrow$ sport için destek $3/7$ değil, $2/7$ olur. $3/7$ yalnız sol tarafın oranıdır ve etikete bağlı değildir.

Hoca konuyu şöyle bağladı: Sınıf birliktelik kurallarında da Apriori ile sık birliktelikler bulunur. Tek fark, kuralın sağ tarafına ürün yerine etiket konmasıdır. Etiket veri setinin içinde bulunur.

> **Hocanın vurgusu:** Sık birliktelikler bulunduktan sonra etiketi sağ tarafa koyarken mutlaka güven değerine bakılır.

19. Sıralı Örüntü Madenciliğine Giriş

Son konu sıralı örüntü madenciliğidir. Hoca bu konunun önemli olduğunu söyledi. Sıralı aksiyonların hangi sırayla alınacağını anlatan bir algoritma türüdür.

Özellikle müşteri kaybı (churn) analizinde ve müşteriyi elde tutma (retention) modellerinde kullanılır. Bir müşterinin sistemi terk edeceğini öğrendiğinizi düşünün. Daha önce terk etme davranışı göstermiş müşterilere uygulanmış sıralı aksiyonlar üzerinde sıralı örüntü madenciliği çalıştırarak, müşteriye en başarılı biçimde ulaşma aksiyonlarını öğrenebilirsiniz. Ayrıntısı gelecek haftaya bırakıldı.

20. Soru-Cevap: Sınıf Birliktelik Kuralları ve TF-IDF

Soru: Sınıf birliktelik kuralı TF-IDF gibi mi çalışıyor?

Cevap: Hayır, odakları farklıdır. TF-IDF bir kelimenin önemini bulmaya yarar. Diyelim elimizde 10 doküman var ve hangi kelimenin önemli olduğuna bakmak istiyoruz. Önce terim sıklığına (term frequency) bakarız: Kelime o dokümanda ne kadar geçmiş? Sonra ters doküman sıklığına (inverse document frequency) bakarız: Kelime 10 dokümanın hepsinde mi, yoksa çok azında mı geçmiş? Hepsinde geçiyorsa o doküman seti için çok ayrıştırıcı gücü yoktur, dolayısıyla çok önemli değildir. Sınıf birliktelik kuralında ise birlikte geçen kelimeleri ve bu kelimelerin konularını (topic) bulursunuz. Yani birinde kelimelerin önemini, diğerinde birliktelikleri bulursunuz.

> **Ders sonrası not:** Hoca bir yerde TF-IDF'in "bir dokümanın" doküman seti içindeki önemini belirttiğini söyledi. Doğrusu, kendisinin aynı cevapta söylediği gibi, bir kelimenin önemidir.

Soru: Buradaki education ve sport etiketleri veri setinde var mı, yoksa çıkarılıyor mu?

Cevap: Etiket mutlaka verilmelidir. Ya açıkça (explicit) ya da dolaylı (implicit) gelir. Açık hali, slayttaki gibi her satırda etiketin yazmasıdır. Dolaylı örnek şudur: "Bu doküman setinde 100 doküman var, 30'u education, 70'i sport. Bundan sınıf birliktelik kuralları çıkar." Bu durumda etiketleri bana söylemiş olur. Her dokümandan bag of words çıkarırım, dokümanın etiketini yanına koyarım ve işlem verisini böyle oluştururum. Sonra sınıf birliktelik kuralı madenciliğini bunun üzerinde çalıştırırım. Sonuçta veri setini slayttaki biçime dönüştürmem gerekir.

Dersin Toparlaması

Bu derste işlem verisinden popüler birliktelikleri ve kuralları çıkarma problemini gördük. Bir kuralın kalitesi iki ölçütle değerlendirilir: Destek, kuralın veride ne kadar yaygın olduğunu, güven ise sol taraf görüldüğünde sağ tarafın ne sıklıkla geldiğini söyler. Apriori bu problemi iki aşamada çözer. Önce düzey düzey ilerleyerek sık öğe kümelerini bulur, aday üretiminde birleştirme ve budama adımlarını kullanır, budamada da aşağı kapalılık özelliğine dayanır. Sonra sık kümelerden $A \rightarrow B$ biçimindeki kuralları yazar ve yalnız güvenlerini hesaplar. Algoritmanın asıl maliyeti, her seviyede veritabanının diskten baştan sona okunmasıdır.

Tablo verisi işlem biçimine çevrilerek aynı algoritmaya verilebilir. Kuralın sağ tarafına etiket konunca da sınıf birliktelik kuralları elde edilir. Dersin sonunda kısaca tanıtılan sıralı örüntü madenciliği gelecek hafta işlenecek.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.